

Reineke-RAG

Technische Dokumentation

Lokales, multi-tenant RAG-System (rag-qdrant-local) · Version 2

100 % offline · on-premise

Stand 21. Mai 2026

Code-Stand: 23 Module · ~5.300 Python-Zeilen · 12 öffentliche HTTP-Endpunkte

1 · Überblick

Reineke-RAG ist ein vollständig **offline** betriebenes Retrieval-Augmented-Generation-System für die Beantwortung natürlichsprachiger Fragen über interne Dokumente (PDF, Word, Excel, PowerPoint, OpenDocument, HTML — auch Legacy-Formate `.doc` und `.xls`). Die aktuelle Codebasis liegt unter `rag-qdrant-local/` und umfasst rund **5.300 Zeilen Python** in einem FastAPI-Backend mit eingebautem Admin-Web-UI.

Eigenschaft	Wert
Einsatzart	Server-seitig, vollständig on-premise
Internet-Zugriff	nicht erforderlich
Quell-Dokumente	Dateisystem + MediaWiki-XML-Import
Unterstützte Formate	PDF, DOCX, DOC, XLSX, XLS, PPTX, ODT, HTML, HTM
Mandantenfähig	ja (Tenant + Project, pro Paar ein „Agent“)
Halluzinationsschutz	Score-Schwelle · Quellenzwang · Meta-Frage-Abfang · Quellen-Trailer-Bereinigung
LLM-Laufzeit	Ollama (Apple-Silicon-tauglich)
Vektor-DB	Qdrant
Reranker	bge-reranker-v2-m3 (Cross-Encoder, lazy, ~2 GB)
Metadaten-Store	SQLite (WAL-Mode) — 10 Tabellen
API-Stil	FastAPI, OpenAPI 3, OpenAI-kompatibel
Admin-UI	Bootstrap 5 + htmx, integriert (<code>/admin/</code>)
Zeitplanung	APScheduler — tägliche Auto-Ingest-Jobs

Das System ist so konzipiert, dass es ausschließlich auf vom Administrator freigegebenen Verzeichnis bäumen arbeitet. Datei-Inhalte verlassen die Maschine nicht. Quell-Verzeichnisse werden **read-only** gelesen.

1.1 Was ist neu in Version 2

Gegenüber dem Auslieferungsstand vom 28. April 2026 sind folgende Bausteine hinzugekommen:

- **Admin-Web-UI** unter `/admin/` — Bootstrap + htmx, 8 Seiten (Übersicht, Agenten, Dokumente, Ingest-Assistent, Zeitplan, Ingest-Verlauf, Logs, Konfiguration).
- **Reranker-Schicht** — BAAI/bge-reranker-v2-m3 Cross-Encoder, lazy geladen, pro Kollektion automatisch ab 100 Dokumenten aktiv (overridebar).
- **Auto-Ingest-Scheduler** — pro Eintrag ein täglicher Cron-Job (HH:MM UTC), integrierter `IngestSchedule`-Tabelle.
- **MediaWiki-Connector** — XML-Export + Uploads + LocalSettings → Pages + Files mit klickbaren Wiki-URLs in den Quellenzitaten.

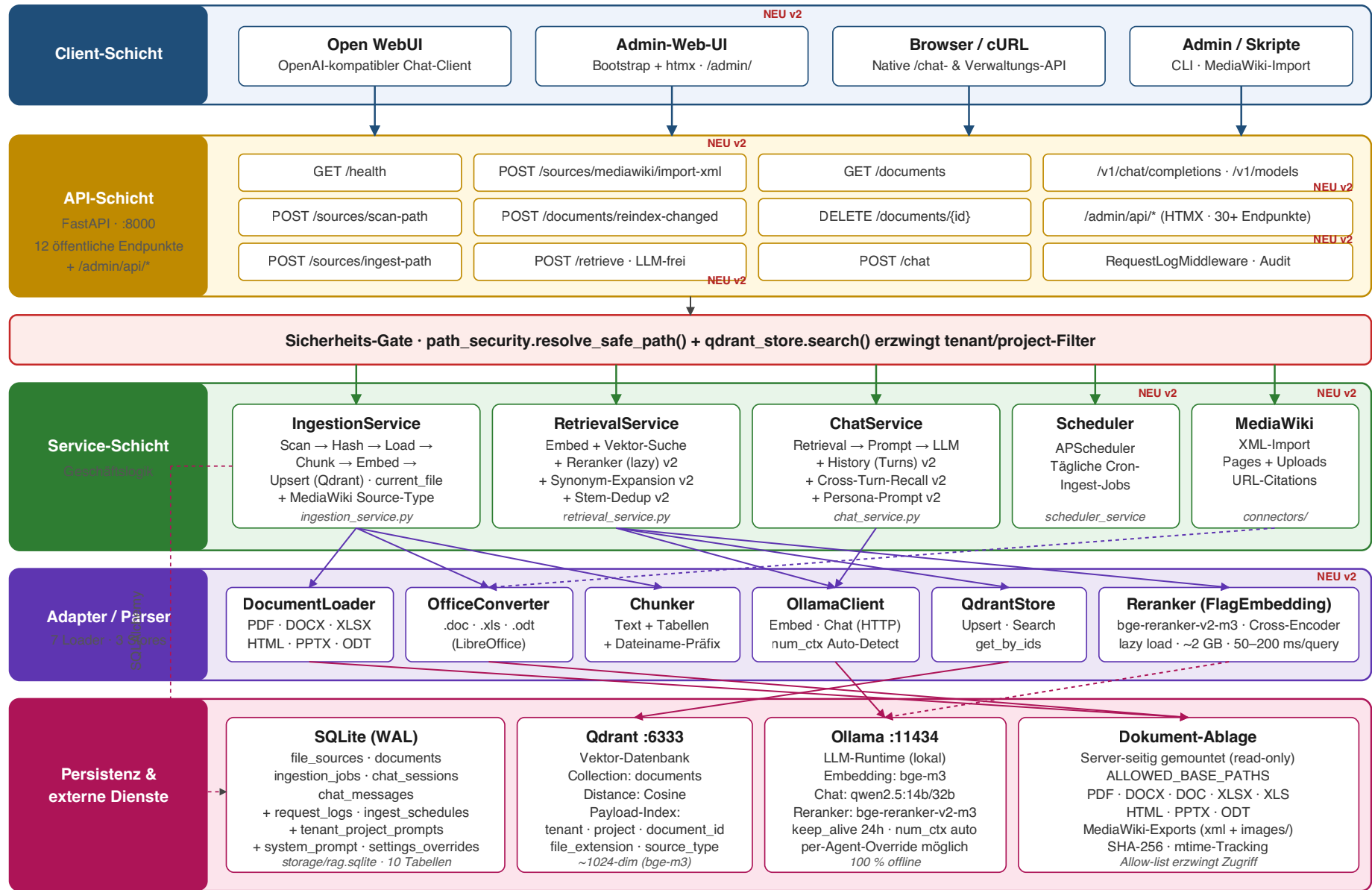
- **Erweiterte Dokumenten-Loader** — neu: PPTX (slide-weise), ODT (über LibreOffice), HTML/HTM (BeautifulSoup, Noise-Stripping, Tabellen als Markdown).
 - **Per-Agent-Persona-Prompts, per-Agent-Chat-Modell, per-Agent-Reranker** in einer eigenen Tabelle `tenant_project_prompts`.
 - **Runtime-Settings-Overrides** — 13 ausgewählte Schlüssel sind ohne Neustart im Admin-UI änderbar (`settings_overrides`-Tabelle).
 - **Globaler System-Prompt-Override** (`system_prompt_overrides`-Tabelle).
 - **Cross-Turn-Citation-Recall** — Zitate früherer Antworten flossen wieder in den Kandidaten-Pool ein, damit Folgefragen wie „die andere Quelle, die du genannt hast“ funktionieren.
 - **Query-Synonym-Expansion** — hand-kuratierte DACH-Synonyme (z. B. Dienstleister ↔ Lieferant) werden vor dem Embedding angehängt.
 - **Stem-basierte Dedup** — identische Dokumente in mehreren Formaten (PDF + DOCX) belegen zusammen nur noch einen Top-K-Platz.
 - **Dateinamen-Präfix in jedem Chunk** — „[Datei: ...]“ am Chunk-Anfang, damit name-zentrische Fragen den richtigen Treffer holen.
 - **Meta-Frage-Abfang** — „wie viele Dokumente?“ liefert die echte SQLite-Zahl statt eines halluzinierten LLM-Werts.
 - **Ollama num_ctx-Auto-Detect** über `/api/show` — kein stilles Truncieren bei Modellen mit ≥ 8 k Kontext.
 - **Pro-Job-Logging + Live-Tail** — Server-Sent-Events streamen Anwendungs- und Request-Logs in das Admin-UI.
 - **Request-Log-Middleware** — Audit-Trail aller HTTP-Anfragen (`request_logs`-Tabelle, Filter im UI).
 - **Strukturierte sources** in OpenAI-Antworten — zusätzlich zum Markdown-Quellenblock, mit `source_type` und `url` (für Wiki-Klicks).
 - **Filename-Junk-Filter** — `~$lock`-Dateien, `._`-Resource-Forks und Zero-Byte Stubs werden beim Scan automatisch ignoriert.
-

2 · Architektur

Das nachfolgende Schema (eine Seite, separat enthalten) zeigt die fünf Schichten — Client, API, Sicherheits-Gate, Service, Adapter/Parser sowie Persistenz/externe Dienste — und die Datenflüsse zwischen ihnen.

Reineke-RAG · System-Architektur (Offline, v2)

Lokale, multi-tenant Retrieval-Augmented-Generation Pipeline · Stand 2026-05-21



■ HTTP-Anfragen

■ Service-Aufrufe (nach Sicherheits-Gate)

■ Adapter-Aufrufe

■ Persistenz / I/O

gestrichelt: nur bei Reranker / nur bei MediaWiki

© Reineke-Technik · alle Daten on-premise

Schichten in Kurzform

1. **Client-Schicht** — Open WebUI · **Admin-Web-UI (NEU)** · Browser/cURL · Admin-Skripte
 2. **API-Schicht** — FastAPI auf Port 8000, native Endpunkte plus OpenAI-Kompatibilität plus `/admin/api/` * (30+ HTMX-Endpunkte)
 3. **Sicherheits-Gate** — Pfad-Allow-list und erzwungene Mandanten-Filter
 4. **Service-Schicht** — `IngestionService`, `RetrievalService`, `ChatService`, `SchedulerService (NEU)`, `MediaWikiImportService (NEU)`
 5. **Adapter & Parser** — Dokumenten-Loader (PDF, DOCX, XLSX, PPTX, ODT, HTML), Office-Konverter, Chunker, Ollama-Client, Qdrant-Store, **Reranker (NEU)**
 6. **Persistenz & externe Dienste** — SQLite, Qdrant (`:6333`), Ollama (`:11434`), Dateisystem
-

3 · Technologie-Komponenten

3.1 Laufzeit & Frameworks

Komponente	Version	Zweck
Python	3.11 (Container) / 3.12 (Tests)	Programmiersprache
FastAPI	0.115.0	HTTP-API, OpenAPI-Spezifikation
Uvicorn	0.30.6	ASGI-Server
Pydantic	2.9.2	Datenvalidierung, Schemata
pydantic-settings	2.5.2	<code>.env</code> -Konfiguration
python-dotenv	1.0.1	Environment-Loader
httpx	0.27.2	Async HTTP-Client (Ollama)
SQLAlchemy	2.0.36	ORM für SQLite
aiosqlite	0.20.0	Async-Treiber
qdrant-client	1.12.1	Qdrant SDK
python-multipart	0.0.12	Multipart-Form-Daten
Jinja2	3.1.4	Admin-UI-Templates (NEU)
APScheduler	3.10.4	Auto-Ingest-Scheduler (NEU)
FlagEmbedding	1.3.4	Cross-Encoder-Reranker (NEU)

3.2 Dokumenten-Parsing

Format	Parser	Strategie
<code>.pdf</code>	pypdf 5.0.1	seitenweise Text-Extraktion, Erkennung Image-only PDFs
<code>.docx</code>	python-docx 1.1.2	Paragraphen + Tabellen → Markdown
<code>.xlsx</code>	openpyxl 3.1.5	header-erkennend, zeilenbasiertes Chunking
<code>.pptx</code>	python-pptx 1.0.2	slide-weise, Tabellen, Speaker Notes (NEU v2)
<code>.odt</code>	LibreOffice → <code>.docx</code>	über <code>office_converter.py</code> (NEU v2)
<code>.html</code> / <code>.htm</code>	BeautifulSoup 4.12.3	Main-Region · Noise-Stripping · Tabellen als Markdown
<code>.doc</code>	LibreOffice → <code>.docx</code>	über <code>office_converter.py</code>
<code>.xls</code>	LibreOffice → <code>.xlsx</code>	über <code>office_converter.py</code>

3.3 KI-Komponenten (lokal)

Aufgabe	Modell-Default	Empfohlen (M4 Max)	Dimension / Parameter
Embedding	<code>mxbai-embed-large</code>	<code>bge-m3</code> (mehrsprachig)	1024
Chat-Generierung	<code>qwen2.5:14b</code>	<code>qwen2.5:32b-instruct-q4_K_M</code>	—
Reranking (NEU)	<code>BAAI/bge-reranker-v2-m3</code>	dito	568 M, lazy geladen

Embedding und Chat laufen über **Ollama** (`http://localhost:11434`) — keine Cloud-Anbindung. Der Reranker läuft als Python-In-Process-Modell (über FlagEmbedding/PyTorch) und wird erst beim ersten Aufruf in den Speicher geladen (5–15 s Cold-Start).

Pro Agent (`(tenant, project)`) lässt sich das Chat-Modell sowie der Reranker-Status über das Admin-UI individuell konfigurieren (`tenant_project_prompts`).

3.4 Datenhaltung

Speicher	Pfad / Endpunkt	Zweck
SQLite	<code>storage/rag.sqlite</code> (WAL)	10 Tabellen — Metadaten, Audit, Chat-Verlauf, Konfiguration
Qdrant	<code>http://localhost:6333</code>	Vektor-Index (Cosine)
Dateisystem	<code>ALLOWED_BASE_PATHS</code>	Original-Dokumente (read-only)
Job-Logs	<code>storage/job-logs/<job_id>.log</code>	Pro-Ingest-Log (rotierend, NEU v2)
Anwendungs-Log	<code>storage/logs/app.log</code>	Rotierend (5x5 MB, NEU v2)

3.5 Container & Deployment

- **Dockerfile** vorhanden (`backend/Dockerfile`) — Python 3.11-slim + LibreOffice + curl.
- **Installer-Bundle** (`installer/`) — `install.sh` , `docker-compose.yml` , systemd-Units, Backup-Timer — siehe Abschnitt 13.
- Externe Dienste (Qdrant, Ollama) werden als Side-Cars / separate Container betrieben.
- Volumes: `storage/rag.sqlite` , `storage/converted/` , `storage/temp/` , `storage/job-logs/` , `storage/logs/` .

4 · Funktionsverzeichnis

Das Backend ist in **23 Python-Module** unter `backend/app/` (+ Admin-UI-Submodul, + Connectors-Submodul) aufgeteilt. Die folgenden Tabellen listen sämtliche öffentlichen Funktionen, Klassen und Methoden auf (strukturiert nach Modul, sortiert nach Aufruftiefe).

4.1 `config.py` — Konfiguration

Symbol	Typ	Zweck
<code>Settings</code>	Klasse	Pydantic- <code>BaseSettings</code> mit allen <code>.env</code> -Werten
<code>Settings.allowed_base_paths</code>	Property	Parst <code>ALLOWED_BASE_PATHS</code> als <code>List[Path]</code>
<code>Settings.sqlite_path</code>	Property	Aufgelöster Pfad zur SQLite-Datei
<code>Settings.converted_dir</code>	Property	Verzeichnis für Office-Konvertierungsergebnisse
<code>Settings.temp_dir</code>	Property	Verzeichnis für temporäre Dateien
<code>Settings.job_logs_dir</code>	Property	Pro-Job-Logverzeichnis (NEU v2)
<code>_project_root() -> Path</code>	Funktion	Ermittelt Projekt-Root unabhängig vom CWD
<code>get_settings() -> Settings</code>	Funktion	Cached Singleton (<code>@lru_cache</code>)

Neue Schlüssel in v2: `OLLAMA_KEEP_ALIVE`, `OLLAMA_NUM_CTX`, `RERANK_ENABLED`, `RERANK_MODEL`, `RERANK_OVERFETCH_K`, `RERANK_AUTO_ENABLE_MIN_DOCS`, `CHAT_HISTORY_TURNS`.

4.2 `database.py` — SQLite Engine & Sessions

Symbol	Signatur	Zweck
<code>_build_engine()</code>	<code>() -> Engine</code>	Erstellt SQLAlchemy-Engine mit WAL + 30-s-Busy-Timeout
<code>init_db()</code>	<code>() -> None</code>	Erstellt alle Tabellen aus <code>models.Base.metadata</code>
<code>session_scope()</code>	<code>() -> Iterator[Session]</code>	Context-Manager mit Commit/Rollback
<code>get_db()</code>	<code>() -> Iterator[Session]</code>	FastAPI-Dependency
<code>SessionLocal</code>	<code>sessionmaker</code>	Für Admin-API-Callers außerhalb von Depends

4.3 `models.py` — ORM-Klassen (10 Tabellen)

Klasse	Tabelle	Wichtige Felder
<code>FileSource</code>	<code>file_sources</code>	<code>id</code> · <code>tenant</code> · <code>project</code> · <code>base_path</code> · <code>recursive</code> · <code>created_at</code> · <code>last_scan_at</code> · <code>last_ingest_at</code>
<code>Document</code>	<code>documents</code>	<code>id</code> · <code>tenant</code> · <code>project</code> · <code>source_path</code> · <code>file_name</code> · <code>file_extension</code> · <code>file_size</code> · <code>checksum</code> · <code>modified_at</code> · <code>status</code> · <code>chunks_count</code> · <code>error_message</code> · <code>source_type</code> · <code>source_metadata_json</code> · <code>created_at</code> · <code>updated_at</code>
<code>IngestionJob</code>	<code>ingestion_jobs</code>	<code>id</code> · <code>tenant</code> · <code>project</code> · <code>source_path</code> · <code>status</code> · <code>files_found</code> · <code>files_indexed</code> · <code>files_skipped</code> · <code>files_failed</code> · <code>chunks_created</code> · <code>current_file</code> · <code>error_message</code> · <code>created_at</code> · <code>completed_at</code>
<code>ChatSession</code>	<code>chat_sessions</code>	<code>id</code> · <code>tenant</code> · <code>project</code> · <code>created_at</code> · <code>messages</code> (1:n)
<code>ChatMessage</code>	<code>chat_messages</code>	<code>id</code> · <code>session_id</code> · <code>role</code> · <code>content</code> · <code>sources_json</code> · <code>created_at</code>
<code>RequestLog</code>	<code>request_logs</code>	<code>id</code> · <code>created_at</code> · <code>method</code> · <code>path</code> · <code>query_string</code> · <code>status_code</code> · <code>duration_ms</code> · <code>tenant</code> · <code>project</code> · <code>client_ip</code> · <code>error_message</code> (NEU v2)
<code>IngestSchedule</code>	<code>ingest_schedules</code>	<code>id</code> · <code>tenant</code> · <code>project</code> · <code>base_path</code> · <code>recursive</code> · <code>reindex_changed_only</code> · <code>hour</code> · <code>minute</code> · <code>enabled</code> · <code>last_run_at</code> · <code>last_status</code> · <code>last_error</code> · <code>last_indexed/skipped/failed/chunks</code> · <code>last_job_id</code> · <code>created_at</code> · <code>updated_at</code> (NEU v2)
<code>TenantProjectPrompt</code>	<code>tenant_project_prompts</code>	PK (<code>tenant</code> , <code>project</code>) · <code>persona_prompt</code> · <code>chat_model</code> · <code>rerank_enabled</code> · <code>rerank_overfetch_k</code> · <code>rerank_model</code> · <code>updated_at</code> (NEU v2)
<code>SystemPromptOverride</code>	<code>system_prompt_overrides</code>	<code>id</code> (<code>'global'</code>) · <code>prompt</code> · <code>updated_at</code> (NEU v2)
<code>SettingsOverride</code>	<code>settings_overrides</code>	<code>key</code> · <code>value</code> · <code>updated_at</code> (NEU v2)

`Document.status` ∈ { `pending`, `indexed`, `failed`, `deleted`, `requires_ocr`, `empty` }.

`Document.source_type` ∈ { `filesystem`, `mediawiki_page`, `mediawiki_upload` }.

4.4 `schemas.py` — Pydantic-DTOs

Schema	Zweck
<code>TenantProject</code>	Mixin für <code>tenant</code> + <code>project</code>
<code>FileEntry</code>	Datei aus Scan-Resultat
<code>HealthCheckItem</code> , <code>HealthResponse</code>	<code>/health</code> -Antwort (jetzt inkl. <code>reranker</code>)
<code>ScanPathRequest</code> , <code>ScanPathResponse</code>	<code>/sources/scan-path</code>
<code>IngestPathRequest</code> (jetzt mit <code>include_extensions</code>), <code>IngestPathResponse</code> , <code>IngestError</code>	<code>/sources/ingest-path</code>
<code>ReindexChangedRequest</code>	<code>/documents/reindex-changed</code>
<code>IngestScheduleIn</code> , <code>IngestScheduleOut</code>	Auto-Ingest-Scheduler (NEU v2)
<code>TenantProjectPromptIn</code>	Persona-Prompt-Pflege (NEU v2)
<code>DocumentOut</code> , <code>DocumentListResponse</code>	<code>GET /documents</code>
<code>DeleteDocumentResponse</code>	<code>DELETE /documents/{id}</code>
<code>ChatRequest</code> , <code>ChatSource</code> (jetzt mit <code>source_type</code> , <code>url</code>), <code>ChatResponse</code>	<code>/chat</code>
<code>RetrieveRequest</code> , <code>RetrieveResponse</code>	<code>/retrieve</code> (LLM-frei, NEU v2)
<code>OpenAIMessage</code> , <code>OpenAIChatCompletionRequest</code> , <code>OpenAIChatCompletionResponse</code> , <code>OpenAIModelEntry</code> , <code>OpenAIModelList</code>	OpenAI-Kompatibilität

4.5 `main.py` — FastAPI-App & Endpunkte

Endpunkt	Methode	Funktion	Beschreibung
<code>/health</code>	GET	<code>health()</code>	Pingt Ollama, Qdrant, Modelle, Reranker (NEU)
<code>/sources/scan-path</code>	POST	<code>sources_scan_path()</code>	Scannt Verzeichnis ohne zu indexieren
<code>/sources/ingest-path</code>	POST	<code>sources_ingest_path()</code>	Vollständiger Ingest mit Embedding + Upsert
<code>/sources/mediawiki/import-xml</code>	POST	<code>sources_mediawiki_import_xml()</code>	MediaWiki-XML-Import (NEU v2)
<code>/documents</code>	GET	<code>list_documents()</code>	Listet indizierte Dokumente
<code>/documents/reindex-changed</code>	POST	<code>reindex_changed()</code>	Reindex nur geänderter Dateien
<code>/documents/{document_id}</code>	DELETE	<code>delete_document()</code>	Löscht Dokument + Vektoren
<code>/chat</code>	POST	<code>chat_endpoint()</code>	RAG-Frage/Antwort mit Quellen
<code>/retrieve</code>	POST	<code>retrieve_endpoint()</code>	LLM-frei — nur Top-K Quellen (NEU v2)
<code>/v1/models</code>	GET	<code>openai_models()</code>	OpenAI-kompatible Modell-Liste
<code>/v1/chat/completions</code>	POST	<code>openai_chat_completions()</code>	OpenAI-kompatibler Chat-Endpunkt
<code>/</code>	GET	Redirect	Redirect auf <code>/admin/</code> (NEU v2)

Lebenszyklus (`lifespan`): `init_db()` → `apply_overrides()` → `num_ctx`-Probe → `scheduler.start()`. Shutdown stoppt den Scheduler.

4.6 `path_security.py` — Pfad-Sicherheit

Symbol	Signatur	Zweck
<code>PathSecurityError</code>	Exception	Eingabe nicht erlaubt
<code>_is_within(child, parent)</code>	<code>(Path, Path) -> bool</code>	Prüft Eltern-Beziehung
<code>_is_under_system_deny(path, allowed_bases)</code>	<code>(Path, List[Path]) -> bool</code>	Schutz vor <code>/etc</code> , <code>/root</code> etc.
<code>get_allowed_base_paths()</code>	<code>() -> List[Path]</code>	Liest und validiert <code>ALLOWED_BASE_PATHS</code>
<code>resolve_safe_path(user_input)</code>	<code>(str) -> Path</code>	Hauptfunktion: kanonisiert + prüft
<code>assert_existing_dir(path)</code>	<code>(Path) -> Path</code>	Prüft Existenz und Verzeichnis-Typ

System-Deny-Liste (Default): `/etc`, `/root`, `/home`, `/var`, `/proc`, `/sys`, `/dev`, `/boot`, `/usr`, `/bin`, `/sbin`, `/lib`, `/opt`.

4.7 `source_scanner.py` — Datei-Scanner

Symbol	Signatur	Zweck
<code>SUPPORTED_EXTENSIONS</code>	Konstante	<code>{.pdf, .docx, .doc, .xls}</code>
<code>_IGNORED_DIR_NAMES</code>	Konstante	<code>.git</code> , <code>__pycache__</code> , <code>node_</code>
<code>_IGNORED_FILE_PREFIXES</code>	Konstante	<code>~\$</code> , <code>._</code> , <code>.</code> (NEU v2 — Office-)
<code>ScanResult</code>	Dataclass	Felder: <code>supported</code> , <code>unsupport</code>
<code>ScanResult.filter_to_extensions(allowed)</code>	Methode	Whitelist-Filter (NEU v2)
<code>MediaWikiExportHint</code>	Dataclass	XML-Pfad + Uploads + LocalSetti
<code>_iter_files(root, recursive)</code>	Iterator	Datei-Iteration mit Ignore-Liste
<code>_is_junk_file(p)</code>	<code>(Path) -> bool</code>	NEU v2 — überspringt Office-Lo
<code>_classify(path)</code>	<code>(Path) -> tuple[bool, str]</code>	(unterstützt?, Endung)
<code>detect_mediawiki_export(root)</code>	<code>(Path) -> Optional[MediaWikiExportHint]</code>	NEU v2 — <code><mediawiki</code> -Sniff +
<code>scan_directory(root, *, recursive=True)</code>	<code>(Path) -> ScanResult</code>	Hauptfunktion

4.8 `document_loader.py` — Dokumenten-Parsing

Symbol	Signatur	Zweck
<code>DocumentLoadError</code>	Exception	Allgemeiner Lade-Fehler
<code>RequiresOCRError</code>	Exception	PDF ohne Text-Layer (Bild-PDF)
<code>LoadedSegment</code>	Dataclass	Ein Textblock + Metadaten
<code>LoadedDocument</code>	Dataclass	Sammlung aller Segmente eines Dokuments
<code>_load_pdf(path)</code>	<code>(Path) -> List[LoadedSegment]</code>	Seitenweises Extrahieren
<code>_table_to_markdown(table)</code>	<code>(table) -> str</code>	Word-Tabelle als Markdown
<code>_load_docx(path)</code>	<code>(Path) -> List[LoadedSegment]</code>	Paragrafen + Tabellen
<code>_load_xlsx(path)</code>	<code>(Path) -> List[LoadedSegment]</code>	Sheet-Erkennung, Header-Heuristik
<code>_load_pptx(path)</code>	<code>(Path) -> List[LoadedSegment]</code>	Slide-weise + Tabellen + Speaker Notes (NEU v2)
<code>_load_odt(path)</code>	<code>(Path) -> List[LoadedSegment]</code>	LibreOffice → docx (NEU v2)
<code>_html_table_to_markdown(table)</code>	<code>(bs4 Tag) -> str</code>	HTML-Tabelle als Markdown
<code>_load_html(path)</code>	<code>(Path) -> List[LoadedSegment]</code>	BeautifulSoup-Parsing, Noise-Filter, Tabellen-Extraktion
<code>load_document(path)</code>	<code>(Path) -> LoadedDocument</code>	Dispatcher für alle Formate

4.9 `office_converter.py` — Legacy-Office-Konvertierung

Symbol	Signatur	Zweck
<code>OfficeConversionError</code>	Exception	LibreOffice-Fehler / fehlt
<code>libreoffice_available()</code>	<code>() -> bool</code>	Prüft <code>soffice</code> im PATH
<code>_run_soffice(src, target_format, outdir)</code>	<code>(Path, str, Path) -> Path</code>	Subprocess-Wrapper, 180 s Timeout
<code>convert_doc_to_docx(src, outdir=None)</code>	<code>(Path) -> Path</code>	<code>.doc</code> → <code>.docx</code>
<code>convert_xls_to_xlsx(src, outdir=None)</code>	<code>(Path) -> Path</code>	<code>.xls</code> → <code>.xlsx</code>
<code>convert_odt_to_docx(src, outdir=None)</code>	<code>(Path) -> Path</code>	<code>.odt</code> → <code>.docx</code> (NEU v2)

4.10 chunker.py — Chunking-Strategien

Symbol	Signatur	Zweck
<code>Chunk</code>	Dataclass	Embed-fähiger Textblock + Metadaten
<code>_filename_prefix(file_name)</code>	<code>(str) -> str</code>	„[Datei: ...]“-Header (NEU v2)
<code>_split_paragraphs(text)</code>	<code>(str) -> List[str]</code>	Trennung an Leerzeilen
<code>_split_long_paragraph(p, size)</code>	<code>(str, int) -> List[str]</code>	Satz-bewusster Hard-Wrap
<code>_chunk_text(text, size, overlap)</code>	<code>(str, int, int) -> List[str]</code>	Sliding Window
<code>_chunk_text_segment(seg, start_index)</code>	<code>(LoadedSegment, int) -> List[Chunk]</code>	Wrapper für Text-Segmente
<code>_xlsx_rows_per_chunk(num_columns)</code>	<code>(int) -> int</code>	Adaptive Block-Größe
<code>_chunk_spreadsheet_segment(seg, start_index)</code>	<code>(LoadedSegment, int) -> List[Chunk]</code>	Tabellen-Chunks (Header wiederholt)
<code>_enforce_char_budget(block, header, budget)</code>	<code>(...) -> List[tuple]</code>	XLSX-Hardcap
<code>chunk_document(doc)</code>	<code>(LoadedDocument) -> List[Chunk]</code>	Dispatcher

Default-Werte: `CHUNK_SIZE=1000`, `CHUNK_OVERLAP=150`, `XLSX_ROWS_PER_CHUNK=40`, `XLSX_MAX_CHARS_PER_CHUNK=6000`.

4.11 `ollama_client.py` — LLM-Client

Methode	Signatur	Zweck
<code>OllamaError</code>	Exception	API- oder Netzwerk-Fehler
<code>OllamaClient.__init__(base_url=None, timeout=600.0)</code>	Konstruktor	DI über Settings
<code>_request(method, path, *, json=None)</code>	async	Low-level HTTP, 1 Retry bei transienten Drops (NEU v2)
<code>list_models()</code>	<code>() -> List[str]</code>	GET <code>/api/tags</code>
<code>has_model(name)</code>	<code>(str) -> bool</code>	Prefix-Match
<code>embed(text, *, model=None)</code>	<code>(str) -> List[float]</code>	POST <code>/api/embeddings</code> mit <code>keep_alive</code>
<code>embed_many(texts, *, model=None)</code>	<code>(List[str]) -> List[List[float]]</code>	Sequenzielle Calls
<code>get_context_length(model)</code>	async, <code>(str) -> int</code>	NEU v2 — <code>/api/show</code> -Probe, cached
<code>chat(messages, *, model=None, temperature=None, max_tokens=None)</code>	<code>(...) -> str</code>	POST <code>/api/chat</code> mit aufgelöstem <code>num_ctx</code>
<code>ping()</code>	<code>() -> bool</code>	Health-Probe

4.12 `qdrant_store.py` — Vektor-Store

Methode	Signatur	Zweck
<code>QdrantStoreError</code>	Exception	Inkonsistenter Aufruf / API-Fehler
<code>SearchHit</code>	Dataclass	<code>score</code> , <code>payload</code> , <code>point_id</code>
<code>QdrantStore.__init__(url=None, api_key=None, collection=None)</code>	Konstruktor	DI über Settings
<code>ping()</code>	<code>() -> bool</code>	Erreichbarkeit
<code>ensure_collection(vector_size)</code>	<code>(int) -> None</code>	Erstellt oder prüft Collection (Cosine)
<code>_ensure_payload_indexes()</code>	<code>() -> None</code>	Idempotente Index-Erstellung (tenant, project, document_id, file_extension)
<code>upsert_chunks(*, document_id, vectors, payloads)</code>	<code>(...) -> int</code>	Schreibt Punkte (Wait=True)
<code>delete_document(document_id)</code>	<code>(str) -> int</code>	Löscht alle Punkte eines Dokuments
<code>search(*, tenant, project, query_vector, top_k, score_threshold=None)</code>	<code>(...) -> List[SearchHit]</code>	Pflicht-Filter: tenant + project, sonst Exception
<code>get_points_by_ids(*, tenant, project, point_ids)</code>	<code>(...) -> List[SearchHit]</code>	NEU v2 — Recall früherer Zitate

4.13 `reranker.py` — Cross-Encoder-Reranker (NEU v2)

Methode	Signatur	Zweck
<code>RerankerError</code>	Exception	Modell-Lade-Fehler, Caller fällt auf Bi-Encoder zurück
<code>_RerankerWrapper.__init__(model_name)</code>	Konstruktor	Lädt FlagReranker, ~2 GB, fp16
<code>_RerankerWrapper.score(query, passages)</code>	<code>(...) -> List[float]</code>	Normalisierte Cross-Encoder-Scores
<code>get_reranker(model_name=None)</code>	<code>(str?) -> _RerankerWrapper</code>	Lazy Singleton pro Modellname
<code>is_loaded(model_name=None)</code>	<code>(str?) -> bool</code>	Reranker-Cold-Start-Indikator (für <code>/health</code>)
<code>loaded_model_names()</code>	<code>() -> List[str]</code>	Aktuell residente Modelle
<code>rerank(*, query, passages, model_name=None)</code>	<code>(...) -> List[float]</code>	Öffentliche Funktion — gibt Scores in Eingangsreihenfolge zurück

4.14 rerank_settings.py — Effektive Reranker-Einstellungen (NEU v2)

Methode	Signatur	Zweck
<code>EffectiveRerankSettings</code>	Dataclass	<code>enabled</code> , <code>overfetch_k</code> , <code>model</code> + Quellen-Marker
<code>smart_default_overfetch_k(doc_count)</code>	<code>(int) -> int</code>	Bucketed: 15 / 30 / 50 / 100
<code>smart_default_rerank_enabled(doc_count)</code>	<code>(int) -> bool</code>	Auto ab <code>RERANK_AUTO_ENABLE_MIN_DOCS</code>
<code>count_indexed_documents(db, *, tenant, project)</code>	<code>(...) -> int</code>	Doc-Count für Auto-Logik
<code>resolve(db, *, tenant, project)</code>	<code>(...) -> EffectiveRerankSettings</code>	Drei-Schichten-Resolver (Override → Smart-Default → Global)

4.15 ingestion_service.py — Ingest-Pipeline

Methode	Signatur	Zweck
<code>IngestionService.__init__(ollama=None, store=None)</code>	Konstruktor	DI
<code>_ensure_collection_for_model()</code>	async	Probt Embedding-Dimension, erstellt Collection
<code>_upsert_file_source(...)</code>	sync	Legt/aktualisiert <code>FileSource</code> -Datensatz
<code>_find_or_create_document(...)</code>	sync	(Document, is_new)
<code>_update_job_progress(...)</code>	static	Schreibt laufende Zähler in <code>IngestionJob</code> (NEU v2)
<code>ingest_path(db, *, tenant, project, path, recursive=True, reindex_changed_only=True, include_extensions=None, job_id=None)</code>	async	Komplette Pipeline; <code>include_extensions=None</code> → alle, leere Liste → nichts; <code>job_id</code> für UI-Live-Progress (NEU v2)
<code>_index_one(*, file_path, document, new_checksum)</code>	async	Pfad → Loaded → delegate
<code>_index_loaded_document(*, document, loaded, checksum, payload_extras=None)</code>	async	NEU v2 — Connector-Einstieg ohne Tempfile
<code>_build_payloads(*, document, chunks, checksum, payload_extras=None)</code>	static	Qdrant-Payload-Aufbau
<code>reindex_changed(db, ...)</code>	async	Differenz-Reindex
<code>delete_document(db, document_id)</code>	async	Qdrant + SQLite löschen

4.16 `retrieval_service.py` — Suche + Reranking

Methode	Signatur	Zweck
<code>_document_stem(file_name)</code>	<code>(str) -> str</code>	Stem für Dedup (NEU v2)
<code>_dedup_by_stem(hits)</code>	<code>(List[SearchHit]) -> List[SearchHit]</code>	PDF+DOCX-Duplikate kollabieren (NEU v2)
<code>_merge_unique_by_point_id(primary, extras)</code>	<code>(...) -> List</code>	Cross-Turn-Recall-Merge (NEU v2)
<code>_expand_query_with_synonyms(question)</code>	<code>(str) -> str</code>	DACH-Synonyme anhängen (NEU v2)
<code>RetrievalService.__init__(ollama, store, session_factory, rerank_fn)</code>	Konstruktor	DI
<code>_load_rerank_fn()</code>	sync	Lazy-Loader für Reranker (NEU v2)
<code>_resolve_rerank(*, tenant, project)</code>	sync	Per-Agent-Resolver (NEU v2)
<code>retrieve(*, tenant, project, question, top_k, min_score, rerank_override, past_citation_ids)</code>	<code>async, () -> List[SearchHit]</code>	Embed + Search + (Cross-Turn-Recall + Rerank) + Dedup + Top-K

4.17 `chat_service.py` — Chat-Orchestrierung

Methode	Signatur	Zweck
<code>DEFAULT_SYSTEM_PROMPT</code>	Konstante	Deutsche RAG-Anweisung (vom DB-Override überschreibbar)
<code>SYSTEM_PROMPT</code>	Alias	Backwards-Compat (für Tests/Previews)
<code>NO_CONTEXT_ANSWER</code>	Konstante	Fallback bei 0 Treffern
<code>hits_to_sources(hits)</code>	Funktion	NEU v2 — Top-Level, von <code>/retrieve</code> mitgenutzt
<code>ChatService.__init__(retrieval, ollama, session_factory)</code>	Konstruktor	DI
<code>_resolve_session_id(...)</code>	sync	DB-Phase A — Session laden/anlegen
<code>_load_persona(tenant, project)</code>	sync	Persona-Prompt aus DB (NEU v2)
<code>_load_chat_model(tenant, project)</code>	sync	Per-Agent-Chat-Modell (NEU v2)
<code>_compose_system_prompt(tenant, project, persona, global_prompt)</code>	static	Globaler Prompt + Persona-Block (NEU v2)
<code>_load_recent_messages(session_id, *, turns)</code>	sync	History-Loader (NEU v2)
<code>_is_meta_count_question(question)</code>	static	Erkennt „wie viele Dokumente?“ (NEU v2)
<code>_meta_count_answer(tenant, project)</code>	sync	Echte SQLite-Zahl (NEU v2)
<code>_load_recent_citation_ids(session_id, *, turns, per_turn)</code>	sync	Cross-Turn-Recall-IDs (NEU v2)
<code>_persist_messages(...)</code>	sync	DB-Phase C — User+Assistant persistieren
<code>_build_context(hits)</code>	static	Quellenblock aufbauen
<code>_format_sources_block(sources)</code>	static	NEU v2 — deterministische Quellen-Sektion (mit URL für Wiki)
<code>_strip_llm_sources_trailer(answer)</code>	static	NEU v2 — Halluzinierte „Quellen:“ entfernen
<code>_ensure_sources_appended(answer, sources_block)</code>	static	Final-Trailer setzen (NEU v2)
<code>chat(*, tenant, project, question, session_id, top_k)</code>	async, <code>() -> ChatResponse</code>	Hauptmethode

4.18 `scheduler_service.py` — Auto-Ingest-Scheduler (*NEU v2*)

Symbol	Signatur	Zweck
<code>IngestSchedulerService.start()</code> / <code>.shutdown()</code>	sync	Lifecycle, hängt an FastAPI-Lifespan
<code>IngestSchedulerService.reload_schedules()</code>	sync	Reconcile In-Memory ↔ DB (idempotent)
<code>IngestSchedulerService.trigger_now(schedule_id)</code>	sync	„Jetzt ausführen“-Button
<code>_run_schedule(schedule_id)</code>	async	Job-Body — Ingest + Outcome zurückschreiben
<code>scheduler</code>	Modul-Singleton	Vom FastAPI-Lifespan importiert

4.19 `system_prompt_store.py` — Globaler System-Prompt (*NEU v2*)

Symbol	Signatur	Zweck
<code>SYSTEM_PROMPT_MAX_CHARS</code>	Konstante	8000 Zeichen Hard-Cap
<code>get_system_prompt()</code>	<code>() -> str</code>	DB-Override oder Default
<code>set_system_prompt(text)</code>	<code>(str) -> None</code>	Persistiert oder löscht (leer = revert)
<code>clear_system_prompt()</code>	<code>() -> None</code>	Revert auf in-code Default
<code>has_override()</code>	<code>() -> bool</code>	UI-Flag

4.20 `settings_overrides.py` — Runtime-Konfigurations-Layer (*NEU v2*)

Symbol	Signatur	Zweck
<code>EditableKey</code>	Dataclass	Metadaten je Schlüssel (Typ, Range, Help, Warning, options_url)
<code>EDITABLE_KEYS</code>	Liste	13 Schlüssel (Embedding-Modell, Chat-Modell, Keep-Alive, Chunks, Retrieval, Temperatur, ...)
<code>_coerce(meta, raw)</code>	sync	Typ-Konversion + Validierung
<code>_apply_to_settings(key, value)</code>	sync	Lebende <code>settings</code> mutieren (+ Logging-Reconfig)
<code>apply_overrides()</code>	sync	Beim Start alle Overrides laden
<code>set_override(key, raw_value)</code>	sync	UI-Speichern
<code>clear_override(key)</code>	sync	UI-Reset
<code>has_override(key)</code>	sync	UI-Flag
<code>overlay_view()</code>	sync	UI-Build (mit override-Markierung)

Infrastrukturschlüssel (`ALLOWED_BASE_PATHS` , `OLLAMA_BASE_URL` , `QDRANT_*` , `HOST` , `PORT` , `SQLITE_DB_PATH` , `SOFFICE_BIN`) sind **nicht** über das UI editierbar — sie erfordern einen Service-Neustart.

4.21 `openai_compat.py` — OpenAI-Adapter

Methode	Signatur	Zweck
<code>OpenAIRequestError</code>	Exception	Anfrage nicht mappbar
<code>ResolvedRequest</code>	Dataclass	tenant, project, question, session_id, model
<code>resolve_openai_request(req)</code>	<code>(...) -> ResolvedRequest</code>	Mapping inkl. <code>extra_body</code> und <code>rag:tenant:project</code> -Modell-ID
<code>build_openai_response(*, model, answer, sources, session_id)</code>	<code>(...) -> OpenAIChatCompletionResponse</code>	Antwort-Aufbau

4.22 `utils.py` — Hilfsfunktionen

Funktion	Signatur	Zweck
<code>configure_logging()</code>	<code>() -> None</code>	Initialisiert Logging laut <code>LOG_LEVEL</code> (inkl. Rotating-File-Handler)
<code>get_logger(name)</code>	<code>(str) -> Logger</code>	Wrapper
<code>new_id()</code>	<code>() -> str</code>	UUID4
<code>utcnow_iso()</code>	<code>() -> str</code>	ISO-8601-Zeitstempel
<code>deterministic_uuid(*parts)</code>	<code>(...) -> str</code>	UUID5 (für Chunk-Point-IDs)
<code>sha256_file(path, *, chunk_size=1MB)</code>	<code>(Path) -> str</code>	Stream-Hash
<code>file_modified_iso(path)</code>	<code>(Path) -> str</code>	mtime als ISO
<code>chunked(items, n)</code>	<code>(...) -> Iterator</code>	Batch-Iterator
<code>capture_logs_for_job(job_id)</code>	Context-Manager	NEU v2 — pro-Job-Filehandler (ContextVar-isoliert)
<code>job_log_path(job_id)</code>	<code>(str) -> Path</code>	Pfad zur Job-Logdatei (NEU v2)

4.23 `admin/` — Admin-Web-UI (NEU v2)

Datei	Zweck
<code>routes.py</code>	Server-Side-Rendered HTML-Seiten (8 Stück, jeweils via Jinja2)
<code>api.py</code>	30+ HTMX-Endpunkte unter <code>/admin/api/*</code> (JSON / Partials)
<code>middleware.py</code>	<code>RequestLogMiddleware</code> → schreibt jeden Request in <code>request_logs</code>
<code>log_stream.py</code>	Server-Sent-Events Streams für Anwendungs- und Request-Log
<code>templates/</code>	<code>base.html</code> , 8 Seiten, 12 Partials
<code>static/css/</code>	<code>bootstrap.min.css</code> + <code>admin.css</code> (Reineke-Overrides)
<code>static/js/</code>	<code>bootstrap.bundle.min.js</code> , <code>htmx.min.js</code> , <code>admin.js</code>

Routen-Übersicht:

Pfad	Inhalt
<code>/admin/</code>	Übersicht (Health-Kacheln, Bestands-Statistik, <code>ALLOWED_BASE_PATHS</code>)
<code>/admin/agenten</code>	Mandant/Projekt-Baum mit Pipe-Code, Persona-Prompt, Chat-Modell, Reranker
<code>/admin/dokumente</code>	Filterbare Dokumenten-Tabelle mit Lösch-Button
<code>/admin/ingest</code>	Zwei-Schritt-Assistent: Scannen → Ingest (mit Dateityp-Filter)
<code>/admin/zeitplan</code>	Auto-Ingest-Cron-Editor
<code>/admin/jobs</code>	Ingest-Verlauf inkl. Status, Fehlern, Log-Download
<code>/admin/logs</code>	API-Audit-Log + Anwendungs-Log mit Live-Tail (SSE)
<code>/admin/konfiguration</code>	Runtime-Einstellungen, globaler System-Prompt, Pipe-Function-Quelltext

4.24 `connectors/mediawiki/` — MediaWiki-XML-Connector (NEU v2)

Datei	Zweck
<code>service.py</code>	Orchestriert XML-Parser → Normalizer → Uploads → <code>IngestionService._index_loaded_document</code>
<code>xml_importer.py</code>	Stream-Parser für <code><mediawiki></code> -Dumps (Pages, Revisionen)
<code>normalizer.py</code>	Wikitext → Markdown (Links, Tabellen, Listen, Templates entfernt)
<code>uploads.py</code>	<code>[[File:...]]</code> -Auflösung gegen <code>images/</code> -Tree (hashed oder flach)
<code>localsettings.py</code>	Regex-Parser für <code>\$wgServer</code> / <code>\$wgArticlePath</code> / <code>\$wgScriptPath</code>
<code>cli.py</code>	<code>inspect-xml</code> (Probelauf) und <code>import-xml</code> Subkommandos
<code>schemas.py</code>	<code>MediaWikiPage</code> , <code>MediaWikiFileRef</code> , <code>MediaWikiWikiConfig</code> , <code>NormalizedPage</code>
<code>errors.py</code>	<code>MediaWikiError</code> plus Spezialisierungen

Zwei Dokumenten-Typen: `mediawiki_page` (eine Seite, latest Revision) und `mediawiki_upload` (eine referenzierte Datei). `payload.url` zeigt im Chat-Quellenblock auf die Wiki-URL und ist anklickbar.

5 · HTTP-API im Detail

5.1 Native Endpunkte

GET /health

Liefert Status-Items (`backend` , `ollama` , `qdrant` , `embedding_model` , `chat_model` , `reranker`). Pro Item `{name, ok, detail}` . Der Reranker ist eine **weiche Abhängigkeit** — `ok=true` bei `disabled` , `enabled` , `lazy` oder `loaded` ; `ok=false` nur bei aktivem Fehler.

POST /sources/scan-path

Body: `{tenant, project, path, recursive?}` . Antwort enthält `supported` , `unsupported` , `file_types` und (falls erkannt) `mediawiki_hint` . Indexiert **nichts**.

POST /sources/ingest-path

Body: `{tenant, project, path, recursive?, reindex_changed_only?, include_extensions?}` .
Komplette Pipeline. `include_extensions=null` ⇒ alle Typen; leeres Array ⇒ Dry-Run. Antwort: `{job_id, indexed_files, skipped_unchanged, failed_files, chunks_created, errors[]}` .

POST /sources/mediawiki/import-xml (NEU v2)

Body:

`{tenant, project, xml_path, uploads_path?, wiki{base_url, article_path, script_path}, allowed_namespaces[], include_redirects, include_uploads, reindex_changed_only, dry_run}` .
Antwort: detaillierte Zähler — `pages_seen / indexed / skipped_*` , `files_seen / indexed / skipped_*` , `unresolved_files` , `warnings` , `errors` . `status` = "ok" oder "partial" .

GET /documents?tenant=...&project=...&include_deleted=false

Listet alle Dokumente eines Mandanten/Projekts mit Status, Chunk-Anzahl und `source_type` .

POST /documents/reindex-changed

Wie `ingest-path` , jedoch ohne neue Dateien — nur geänderte werden re-embedded. Optional `mark_missing_as_deleted=true` markiert verschwundene Dokumente.

DELETE /documents/{document_id}

Löscht alle zugehörigen Qdrant-Punkte und setzt den SQLite-Status auf `deleted` (Soft-Delete).

POST /chat

Body: `{tenant, project, question, session_id?, top_k?}` . Antwort: `{answer, sources[], session_id}` . Antwortfluss siehe Abschnitt 6.2.

POST /retrieve (NEU v2)

LLM-frei — nur Top-K-Quellen ohne Generation. Identischer Retrieval-Pfad (Embedder, Score-Schwelle, Reranker, Synonym-Expansion). Wird vom Eval-Runner für Recall@K / MRR-Messungen genutzt.

5.2 OpenAI-kompatible Endpunkte

GET /v1/models

Listet virtuelle Modelle in der Form `rag:<tenant>:<project>` (oder `rag:default`, falls keine Dokumente indiziert sind). Optionale Filter `?tenant=` und `?project=`.

POST /v1/chat/completions

Akzeptiert OpenAI-Standard. Tenant/Projekt werden bestimmt aus 1. Inline-Feldern, 2. `extra_body`, 3. Modell-ID `rag:tenant:project`. Streaming wird abgelehnt (`stream=true` ⇒ Fehler). Response enthält zusätzlich (nicht-Standard, abwärtskompatibel) ein strukturiertes `sources[]`-Array und `session_id`.

5.3 Admin-API (Auswahl)

Unter `/admin/api/*` (nicht in der OpenAPI-Spec exportiert, dient ausschließlich dem mitgelieferten Admin-UI):

Pfad	Zweck
GET /admin/api/health	Health-Karten-Partial
GET /admin/api/tenants / .json	Mandanten-Baum
GET /admin/api/documents	Filterbare Dokumenten-Tabelle
DELETE /admin/api/documents/{id}	Löschen
POST /admin/api/ingest/scan	Scan-Partial (mit Skip-Statistik)
POST /admin/api/ingest/run	Ingest starten (mit Live-Progress-Polling)
POST /admin/api/ingest/run-mediawiki	MediaWiki-Import starten
GET /admin/api/jobs / {id}/progress / {id}/logs / {id}/logs.txt	Ingest-Verlauf
GET /admin/api/schedules · POST · DELETE /{id} · POST /{id}/run-now	Auto-Ingest-Zeitpläne
GET /admin/api/request-logs / /stream	Request-Audit (HTML + SSE)
GET /admin/api/app-log/stream	Anwendungs-Log (SSE)
GET /admin/api/config · POST /{key} · POST /{key}/reset	Runtime-Settings
GET /admin/api/system-prompt · POST · DELETE	Globaler System-Prompt
POST /admin/api/agents/{tenant}/{project}/prompt	Persona-Prompt
POST /admin/api/agents/{tenant}/{project}/chat-model	Per-Agent-Chat-Modell
POST /admin/api/agents/{tenant}/{project}/rerank	Per-Agent-Reranker
GET /admin/api/agents/{tenant}/{project}/prompt-preview	Zusammengesetzter System-Prompt
GET /admin/api/openwebui-pipe.py	Vorkonfigurierte Pipe-Funktion (mit ? tenant=&project= substituiert)
GET /admin/api/ollama/models?role=chat / ? role=embedding	Verfügbare Ollama-Modelle

5.4 Fehler-Klassen

HTTP	Auslöser
400	PathSecurityError , OpenAIRequestError , MediaWikiError , ValueError
404	Dokument / Schedule nicht gefunden
500	QdrantStoreError , allgemeiner Fehler
502	OllamaError (LLM nicht erreichbar)

6 · Datenflüsse

6.1 Ingest-Pipeline (Dateisystem)

1. `POST /sources/ingest-path` → Pfad-Sicherheits-Gate.
2. `IngestionService.ingest_path()` öffnet Qdrant-Collection (Dimension passend zum Embedding-Modell).
3. `scan_directory()` liefert alle unterstützten Dateien (inkl. MediaWiki-Hint).
4. Optionaler `include_extensions` -Filter narrowt die Arbeitsmenge.
5. Pro Datei (in der Schleife, mit live aktualisiertem `IngestionJob.current_file`): * `sha256_file()` berechnet Prüfsumme. * Bei unverändertem Hash + Status `indexed` → Skip. * `load_document()` extrahiert Segmente (PDF, DOCX, XLSX, PPTX, ODT, HTML, ...). * `chunk_document()` erzeugt Chunks (Text- oder Tabellen-Strategie), jeder Chunk bekommt das „[Datei: stem]“-Präfix. * `OllamaClient.embed_many()` liefert Vektoren. * `QdrantStore.upsert_chunks()` schreibt Punkte mit Tenant/Projekt-Payload. * SQLite-Status auf `indexed` (oder `failed` / `requires_ocr` / `empty`). * `_update_job_progress()` schreibt laufende Zähler.
6. Antwort enthält Job-Statistik und Fehlerliste. Der Job-Log liegt unter `storage/job-logs/<job_id>.log` und ist im UI tailbar.

6.2 Chat-Pipeline

1. `POST /chat` (oder `/v1/chat/completions`).
2. `_resolve_session_id()` lädt oder legt eine `ChatSession` an (DB-Phase A, kurzlebig).
3. **Meta-Frage-Abfang:** Falls die Frage nach Mengenangaben („wie viele Dokumente?“) aussieht ⇒ echter `SELECT COUNT(*)` aus SQLite, sofort Antwort, kein LLM-Aufruf.
4. `_load_recent_citation_ids()` holt Qdrant-IDs der letzten drei zitierten Antworten dieser Session (Cross-Turn-Recall).
5. `RetrievalService.retrieve()`: * `_expand_query_with_synonyms()` ergänzt DACH-Synonyme. * `OllamaClient.embed()` erzeugt Frage-Embedding. * `QdrantStore.search()` mit erzwungenem Tenant/Projekt-Filter + Score-Schwelle liefert Overfetch-Kandidaten (`top_k * 2` oder `overfetch_k`). * Cross-Turn-Recall-Punkte werden in den Pool gemerged. * Falls Reranker aktiv (Per-Agent-Override oder Auto): Cross-Encoder reordert, der Score ersetzt den Bi-Encoder-Score. * `_dedup_by_stem()` faltet PDF/DOCX-Dubletten zusammen, dann `[:top_k]`.
6. Bei **0 Treffern** → `NO_CONTEXT_ANSWER`, Persistierung, Rückgabe.
7. Sonst: `_build_context()` erzeugt nummerierten Quellenblock, `_compose_system_prompt()` baut System-Prompt + Persona, `_load_recent_messages()` lädt bis zu `CHAT_HISTORY_TURNS` -Paare, `OllamaClient.chat()` generiert Antwort (mit per-Agent-Chat-Modell, falls gesetzt).
8. `_strip_llm_sources_trailer()` entfernt halluzinierte „Quellen:“-Listen, `_ensure_sources_appended()` hängt unseren deterministischen Quellenblock an.
9. `_persist_messages()` speichert User- und Assistant-Message inkl. `sources_json`.
10. Antwort: `{answer, sources, session_id}`. `sources[].url` enthält bei MediaWiki-Quellen die Wiki-URL.

6.3 Auto-Ingest-Scheduler (NEU v2)

1. FastAPI-Lifespan ruft `scheduler.start()` → `AsyncIOScheduler` läuft im Process.
2. `reload_schedules()` liest alle `IngestSchedule` -Zeilen, registriert je aktivem Eintrag einen `CronTrigger(hour, minute, second=0, timezone='UTC')`.
3. Beim Trigger ruft `_run_schedule(schedule_id)` `IngestionService.ingest_path()` und schreibt `last_run_at`, `last_status`, `last_indexed/skipped/failed/chunks`, `last_job_id` zurück in die Schedule-Zeile.
4. Admin-UI „Jetzt ausführen“ ruft `trigger_now()` → Einmal-Job für sofortigen Lauf.
5. CRUD über die `/admin/api/schedules` -Endpunkte ruft jeweils `reload_schedules()`, sodass die In-Memory-Jobs konsistent mit der DB bleiben.

6.4 MediaWiki-Import (NEU v2)

1. `POST /sources/mediawiki/import-xml` → Pfad-Sicherheits-Gate für `xml_path` und `uploads_path`.
2. `MediaWikiImportService.import_xml()` : * `iter_pages()` stream-parst den XML-Dump, applies Namespace- und Redirect-Filter. * `normalize_wikitext()` macht aus Wikitext lesbares Markdown, extrahiert Kategorien und `[[File:...]]` -Referenzen. * Für jede Seite: `Document` (Typ `mediawiki_page`) mit `source_metadata_json` (page_id, revision_id, categories, linked_files, page_url), `_index_loaded_document()` mit `payload_extras={url: wiki_url, source_type: ...}`. * Zweiter Pass: jedes referenzierte Upload wird via `resolve_upload()` aufgelöst (hashed `a/ab/...` oder flach) und mit dem normalen Datei-Loader ingestet.
3. Antwort enthält detaillierte Zähler und Listen `unresolved_files`, `warnings`, `errors`.

6.5 Externe Service-Abhängigkeiten

Quelle	Ziel	Protokoll	Zweck
Backend	Ollama (:11434)	HTTP	Embeddings, Chat, <code>/api/show</code> (num_ctx-Probe)
Backend	Qdrant (:6333)	HTTP	Upsert, Suche, Retrieve, Indexes
Backend	Dateisystem	POSIX	Originaldokumente lesen, Konvertierungen schreiben
Backend	LibreOffice (Subprozess)	CLI	Legacy-Konvertierung (<code>.doc</code> / <code>.xls</code> / <code>.odt</code>)
Backend	FlagEmbedding / PyTorch	In-Process	Reranker (lazy, lokal)
Browser	Backend <code>/admin/api/*/stream</code>	SSE	Live-Tail Anwendungs- und Request-Log

7 · Datenmodell

7.1 SQLite-Schema (10 Tabellen)

Tabelle	Schlüssel	Zweck
<code>file_sources</code>	id	Konfigurierte Wurzelpfade pro Mandant
<code>documents</code>	id (UNIQUE: tenant, project, source_path)	Indizierte Dateien (+ <code>source_type</code> , <code>source_metadata_json</code>)
<code>ingestion_jobs</code>	id	Audit-Trail aller Ingest-Läufe (+ <code>current_file</code>)
<code>chat_sessions</code>	id	Konversationscontainer
<code>chat_messages</code>	id (FK: session_id)	Einzelne Nachrichten + Quellen
<code>request_logs</code> (NEU)	id	HTTP-Audit für <code>/admin/logs</code>
<code>ingest_schedules</code> (NEU)	id	Wiederkehrender Auto-Ingest pro Agent
<code>tenant_project_prompts</code> (NEU)	PK (tenant, project)	Persona, Chat-Modell-Override, Reranker-Override
<code>system_prompt_overrides</code> (NEU)	id ('global')	Globaler System-Prompt-Override
<code>settings_overrides</code> (NEU)	key	Runtime-überschriebene Settings

WAL-Mode, `foreign_keys=ON`, `synchronous=NORMAL`, `busy_timeout=30 s`.

Indexes über `tenant`, `project`, `status`, `created_at` + `status_code` (für die Request-Log-Filter im UI), plus der zusammengesetzte UNIQUE-Index `(tenant, project, source_path)` auf `documents`.

7.2 Qdrant-Punktformat

```
{
  id      : UUID5(document_id, chunk_index),
  vector  : [float × dim],
  payload : {
    tenant, project, document_id, file_name,
    file_extension, source_path, checksum, modified_at, created_at,
    document_type, chunk_index, page?, sheet?,
    row_start?, row_end?, text,
    source_type,           // filesystem | mediawiki_page | mediawiki_upload
    url?,                 // wiki click-through, optional (NEU v2)
    page_id?, revision_id? // MediaWiki-Provenance (NEU v2)
  }
}
```

Indizierte Payload-Felder: `tenant`, `project`, `document_id`, `file_extension`.

8 · Konfiguration (`.env`)

Variable	Default	Beschreibung
OLLAMA_BASE_URL	http://localhost:11434	LLM-Endpoint
OLLAMA_KEEP_ALIVE	1h	NEU v2 — 5m / 24h / -1 (forever)/ 0
OLLAMA_NUM_CTX	(unset, auto)	NEU v2 — manueller Override des KV-Cache-Fensters
QDRANT_URL	http://localhost:6333	Vektor-DB
QDRANT_API_KEY	—	Optional
QDRANT_COLLECTION	documents	Collection-Name
EMBEDDING_MODEL	mxbai-embed-large	Empfohlen: bge-m3
CHAT_MODEL	qwen2.5:14b	Globaler Fallback (per-Agent-Override im UI)
RERANK_ENABLED	true	NEU v2 — globaler Killswitch
RERANK_MODEL	BAAI/bge-reranker-v2-m3	NEU v2 — Standard-Reranker
RERANK_OVERFETCH_K	20	NEU v2 — Mindest-Kandidatenpool
RERANK_AUTO_ENABLE_MIN_DOCS	100	NEU v2 — Auto-Aktivierungs-Schwelle
ALLOWED_BASE_PATHS	—	Pflicht — kommaseparierte absolute Pfade
SQLITE_DB_PATH	./storage/rag.sqlite	Metadaten-DB
CHUNK_SIZE	1000	Zeichen pro Text-Chunk
CHUNK_OVERLAP	150	Überlappung
XLSX_ROWS_PER_CHUNK	40	Zeilen pro Tabellen-Chunk
XLSX_MAX_CHARS_PER_CHUNK	6000	Hardlimit
RETRIEVAL_TOP_K	6	Anzahl Treffer
MIN_RETRIEVAL_SCORE	0.35	Schwelle für „relevant“
CHAT_TEMPERATURE	0.1	Sampling
CHAT_MAX_TOKENS	1024	Max. Antwortlänge
CHAT_HISTORY_TURNS	6	NEU v2 — Anzahl früherer Frage-/Antwort-Paare im LLM-Kontext
SOFFICE_BIN	soffice	LibreOffice-Binary
HOST	0.0.0.0	FastAPI
PORT	8000	FastAPI
LOG_LEVEL	INFO	DEBUG/INFO/WARN/ERROR

13 dieser Schlüssel sind zusätzlich über das Admin-UI ohne Neustart änderbar (siehe `settings_overrides.EDITABLE_KEYS`). Infrastruktur- und Sicherheitsschlüssel sind bewusst nicht editierbar.

9 · Sicherheits-Eigenschaften

1. **Allow-list-basierter Dateizugriff.** Ohne `ALLOWED_BASE_PATHS` lehnt das System jede Scan- oder Ingest-Anfrage ab. Gilt auch für den MediaWiki-Connector (`xml_path` und `uploads_path` werden vor dem Lesen validiert).
 2. **Pfad-Traversal-Schutz.** `resolve_safe_path()` kanonisiert mit `Path.resolve()`, prüft Eltern-Beziehung, lehnt Systemverzeichnisse (`/etc`, `/root`, ...) ab, sofern diese nicht explizit freigegeben sind.
 3. **Mandanten-Isolation als Pflicht-Filter.** `QdrantStore.search()` und `get_points_by_ids()` werfen `QdrantStoreError`, wenn `tenant` oder `project` leer sind — eine ungefilterte Vektor-Suche ist nicht möglich.
 4. **Anti-Halluzination.** Ohne Treffer über `MIN_RETRIEVAL_SCORE` gibt das System die Konstante `NO_CONTEXT_ANSWER` zurück; das LLM wird in diesem Fall **nicht** angefragt. Meta-Fragen werden direkt aus SQLite beantwortet. LLM-erfundene „Quellen:“-Trailer werden deterministisch entfernt und durch den realen Quellenblock ersetzt.
 5. **Quellenzwang.** Jede generierte Antwort wird mit `sources` (Datei, Seite/Sheet/Zeilenbereich, Score, plus `source_type` und `url` für Wiki-Quellen) ausgeliefert und persistiert.
 6. **Offline-Betrieb.** Sämtliche Modelle laufen lokal über Ollama bzw. lokal geladene Reranker-Gewichte; das Backend macht keine ausgehenden Verbindungen außer zu Qdrant und Ollama.
 7. **Read-only auf Originale.** Ingest-Pipeline schreibt nicht in die Quellverzeichnisse. Konvertierungsergebnisse landen in `storage/converted/`.
 8. **Junk-Filter.** Office-Lock-Dateien (`~$...`), macOS-Resource-Forks (`._...`), sonstige Dotfiles und Zero-Byte-Stubs werden beim Scan ignoriert, bevor sie die Loader erreichen.
 9. **Audit-Trail.** Jede HTTP-Anfrage landet (außer SSE-Streams) in `request_logs` mit Methode, Pfad, Statuscode, Dauer, Tenant/Project und Client-IP.
 10. **Session-zentrierte DB-Schreibphasen.** Während der LLM-Generation hält ChatService keine SQLite-Connection — das schützt den Connection-Pool unter Last und verhindert WAL-Blockaden.
-

10 · Tests

Test-Verzeichnis: `backend/tests/` (25 Module + Eval-Unterverzeichnis). Ausführung:

```
cd backend  
pytest -v
```

Testdatei	Fokus
<code>test_path_security.py</code>	Allow-list, Traversal, System-Pfad-Ablehnung
<code>test_source_scanner.py</code>	Klassifikation, Recursive-Flag, Junk-Filter
<code>test_scanner_mediawiki_detect.py</code>	MediaWiki-Export-Erkennung (NEU v2)
<code>test_extractors.py</code>	PDF/DOCX/XLSX/HTML-Parsing
<code>test_pptx_loader.py</code>	PPTX-Slides + Speaker Notes (NEU v2)
<code>test_chunker_xlsx.py</code>	XLSX-Chunking inklusive Header-Wiederholung
<code>test_chunker_filename.py</code>	Filename-Prefix in jedem Chunk (NEU v2)
<code>test_qdrant_filter.py</code>	Pflicht-Filter Tenant + Projekt
<code>test_openai_compat.py</code>	OpenAI-Request-Auflösung
<code>test_chat_no_hits.py</code>	Fallback bei fehlenden Treffern (async)
<code>test_chat_session_lifecycle.py</code>	Session-IDs, Tenant/Projekt-Cross-Check (NEU v2)
<code>test_chat_cross_turn_recall.py</code>	Past-Citation-IDs (NEU v2)
<code>test_chat_meta_question.py</code>	„wie viele Dokumente?“ (NEU v2)
<code>test_chat_sources_trailer.py</code>	Halluzinierte Quellen entfernen (NEU v2)
<code>test_retrieve_endpoint.py</code>	LLM-freier Retrieve-Endpoint (NEU v2)
<code>test_retrieval_rerank.py</code>	Reranker-Pfad inkl. Fallback (NEU v2)
<code>test_retrieval_dedup.py</code>	Stem-Dedup (NEU v2)
<code>test_rerank_settings.py</code>	Drei-Schichten-Resolver (NEU v2)
<code>test_query_synonym_expansion.py</code>	Dienstleister ↔ Lieferant (NEU v2)
<code>test_ollama_num_ctx.py</code>	<code>/api/show</code> -Probe + Cache (NEU v2)
<code>test_settings_overrides.py</code>	Runtime-Editieren, Coercion (NEU v2)
<code>test_system_prompt_override.py</code>	DB-Override + Revert (NEU v2)
<code>test_persona_prompt.py</code>	Persona je Mandant/Projekt (NEU v2)
<code>test_ingest_progress.py</code>	<code>current_file</code> , Prozent, ETA (NEU v2)
<code>test_job_logs.py</code>	Pro-Job-Logging (NEU v2)
<code>connectors/mediawiki/...</code>	XML, Normalizer, Uploads, Service, Integration
<code>eval/</code>	Versionierter Fragenkatalog (Recall@5 / MRR / Faithfulness / Latenz)

CI führt die Suite auf jedem PR aus (Python 3.11 + 3.12, GitHub Actions).

11 · Admin-Web-UI

Das Admin-UI ist Server-Side-Rendered (Jinja2) und nutzt **htmx** für partielle Updates — keine SPA, kein Build-Schritt, kein npm. Statische Assets (Bootstrap, htmx) liegen direkt im Repository unter `backend/app/admin/static/`.

Seite	Routine im Backend	Wichtigste Inhalte
Übersicht	<code>routes.page_overview()</code>	Health-Karten (alle 15 s), Doc-/Chunk-/Agent-Zähler, Allow-list
Agenten	<code>routes.page_agents()</code>	Mandant→Projekt-Baum, Pipe-Function-Quelltext, Persona-Prompt-Editor, Chat-Modell-Picker, Reranker-Tri-State
Dokumente	<code>routes.page_documents()</code>	Filterbare Tabelle (Mandant/Projekt/Status/Suche), Lösch-Button
Ingest-Assistent	<code>routes.page_ingest()</code>	Zwei-Schritt-Wizard: Scannen → Dateityp-Auswahl → Ingest (mit Live-Progress)
Zeitplan	<code>routes.page_schedules()</code>	Cron-Editor (HH:MM UTC), aktive/pausierte Einträge, „Jetzt ausführen“
Ingest-Verlauf	<code>routes.page_jobs()</code>	Letzte 200 Läufe mit Status, Fehlerindikator, Log-Download
Logs	<code>routes.page_logs()</code>	API-Audit (Filter: Status/Methode/Mandant) und Anwendungs-Log mit Live-Tail (SSE)
Konfiguration	<code>routes.page_config()</code>	Runtime-Settings, Read-only-Infra-Felder, globaler System-Prompt, Pipe-Function-Quelltext zum Kopieren

Die UI nutzt durchweg **deutsche Beschriftungen**. Das `data-bs-theme="dark"` gilt nur für die Navbar; Inhaltsbereiche sind im Light-Modus.

Sicherheit: Das UI ist **nicht** durch Authentifizierung geschützt. Es lebt im Vertrauensmodell „intern erreichbar, Reverse-Proxy macht TLS und ggf. Auth“. Für externe Exposition muss vor dem Backend ein Auth-Proxy stehen (zukünftiger OIDC-Support ist im Installer-README erwähnt).

12 · Bedienung über die HTTP-API

12.1 Voraussetzungen

- Ollama $\geq$ 0.4 lokal laufend (Modelle vorgepullt: `bge-m3` , `qwen2.5:14b/32b` o. ä.).
- Qdrant $\geq$ 1.12 erreichbar.
- LibreOffice (für `.doc` / `.xls` / `.odt`) — optional, falls Legacy-Formate vorkommen.
- Reranker-Dependencies (PyTorch via FlagEmbedding) sind im Image enthalten, belegen Disk aber nicht VRAM, solange `RERANK_ENABLED=false` .

12.2 Schnellstart

```
git clone <repo>
cd rag-qdrant-local
cp .env.example .env
# In .env: ALLOWED_BASE_PATHS=/srv/dokumente,/mnt/policies ← anpassen

cd backend
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Anschließend ist das Admin-UI unter `http://localhost:8000/admin/` erreichbar (siehe Handbuch). API-Dokumentation: `http://localhost:8000/docs` .

12.3 Beispiel-Workflow (CLI)

```
# 1. Health-Check
curl -s http://localhost:8000/health | jq

# 2. Scan (zeigt Dateitypen ohne zu indexieren)
curl -X POST http://localhost:8000/sources/scan-path \
  -H "Content-Type: application/json" \
  -d '{"tenant":"reineke","project":"watch","path":"/srv/dokumente/watch"}'

# 3. Ingest (komplett, oder nur ausgewählte Endungen)
curl -X POST http://localhost:8000/sources/ingest-path \
  -H "Content-Type: application/json" \
  -d '{
    "tenant":"reineke","project":"watch",
    "path":"/srv/dokumente/watch",
    "include_extensions":[".pdf",".docx"]
  }'

# 4. Frage stellen
curl -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{
    "tenant":"reineke","project":"watch",
    "question":"Welche Passwortregeln gelten?"
  }'

# 5. LLM-freier Retrieve (nur Quellen, für Eval-Runs)
curl -X POST http://localhost:8000/retrieve \
  -H "Content-Type: application/json" \
  -d '{
    "tenant":"reineke","project":"watch",
    "question":"Welche Passwortregeln gelten?","top_k":5
  }'
```

12.4 Open-WebUI-Integration

- In Open WebUI als „OpenAI-API“ eintragen, Base-URL = `http://<backend>:8000/v1`.
- API-Key beliebig (wird nicht geprüft).
- Modell wählen: `rag:reineke:watch` (Format `rag:<tenant>:<project>`).
- Streaming **aus** (`stream=true` wird abgelehnt).
- Alternativ die im Admin-UI angebotene Pipe-Function nutzen — sie ist pro `(tenant, project)`-Paar vorbefüllt und kopierbar.

12.5 Wartung

Aktion	Befehl
Reindex geänderter Dateien	<code>POST /documents/reindex-changed</code>
Verschwundene Dateien als „deleted“	<code>...?mark_missing_as_deleted=true</code>
Einzelnes Dokument entfernen	<code>DELETE /documents/{id}</code> (auch im UI per Klick)
SQLite-Backup	<code>sqlite3 storage/rag.sqlite ".backup backup.sqlite"</code>
Qdrant-Snapshot	<code>curl -X POST http://localhost:6333/collections/documents/snapshots</code>
Auto-Ingest pausieren	UI: Zeitplan → Eintrag deaktivieren
Settings ohne Neustart anpassen	UI: Konfiguration → Wert ändern → „↵“

13 · Auslieferung & Installer-Bundle

Reineke-RAG wird als versioniertes Installer-Bundle ausgeliefert (`scripts/build-bundle.sh`):

```
reineke-rag-installer-<version>.tar.gz
├── README.md           # Installer-Anleitung
├── install.sh          # Haupt-Installer (Docker Compose + systemd)
├── uninstall.sh
├── docker-compose.yml  # Qdrant + Ollama + Backend
├── .env.example
├── images/             # (offline) Container-Tarballs
├── models/             # (offline) Ollama-Modell-Blobs
├── scripts/            # wait-for / backup / restore
├── systemd/           # reineke-rag.service + Backup-Timer
└── VERSION
```

Der Installer:

1. prüft Voraussetzungen (Docker, Compose v2),
2. legt `/opt/reineke-rag/` an,
3. lädt Images (offline aus `images/` oder online),
4. startet Qdrant + Ollama,
5. pullt / lädt KI-Modelle (`bge-m3` , Chat-Modell, optional Reranker),
6. startet das Backend, ruft `GET /health` ab,
7. installiert systemd-Units (Reboot-fest),
8. richtet täglichen Backup-Timer ein (03:00, Aufbewahrung 14 Tage).

Apple-Silicon-Spezialfall: Ollama läuft **nativ** auf dem Host (deutlich schneller als die Linux-VM-Variante); der Installer wird mit `--skip-ollama` aufgerufen.

14 · Status & nächste Schritte

- **Reife.** End-to-End funktionsfähig; alle 25 Test-Module grün. Retrieval-Quality-Eval (`tests/eval/`) liefert Recall@5 = 11/11 (Stand 2026-05-20, Korpus 62 Dokumente).
- **Empfohlene Konfiguration für DACH-Kunden.**
 - Embedder: `bge-m3` (Default in `.env.example` bereits gesetzt).
 - Chat: `qwen2.5:32b-instruct-q4_K_M` (Qualität) oder `qwen2.5:14b` (Throughput).
 - Reranker: globalen Killswitch auf `true` , Smart-Default aktivieren — pro Kollektion ab 100 Dokumenten automatisch an.
- **Skalierung.** SQLite-Setup (Busy-Timeout 30 s, WAL) ist für moderate Parallellast (≤ 10 gleichzeitige Chats) ausgelegt; bei höherer Last Migration auf PostgreSQL.
- **Offene Themen.**
 - OIDC / SSO vor das Admin-UI schalten.
 - MediaWiki SQL-Mode (`POST /sources/mediawiki/import-sql`) — Design vorhanden, teilt die Service-Schicht mit dem XML-Mode.
 - OCR-Pipeline für reine Bild-PDFs (heute werden sie als `requires_ocr` markiert).

Diese Dokumentation wurde aus der Codebasis `rag-qdrant-local/` (rund 5.300 Python-Zeilen, 23 Module, 12 öffentliche HTTP-Endpunkte plus ein Admin-API-Submodul mit über 30 HTMX-Endpunkten) erzeugt. Für Detailfragen siehe Inline-Docstrings, das Handbuch (`HANDBUCH.pdf`) und `TEST_REPORT.md` .